

Quadro Vivente: Relazione del Progetto di Laboratorio di Making

Giovanni Maria Savoca
giovannimaria.savoca@studio.unibo.it

29 maggio 2025

Sommario

Questa relazione descrive il progetto “Quadro Vivente”, realizzato per il corso di Laboratorio di Making della Laurea Magistrale in Informatica presso l’Università di Bologna (UniBo). L’obiettivo è mettere in pratica competenze di elettronica, programmazione embedded e sviluppo web, trasformando dati ambientali in un’esperienza visiva interattiva. Si illustrano l’architettura hardware/software, il meccanismo di comunicazione server-client e il percorso di sviluppo, dall’ideazione alla realizzazione finale.

1 Introduzione

Il progetto “Quadro Vivente” nasce dall’idea di trasformare dati ambientali in un’esperienza visiva dinamica. Utilizzando sensori di temperatura, umidità, luce e suono, il sistema genera in tempo reale immagini naturalistiche che variano in base alle condizioni rilevate.

L’ispirazione è arrivata riflettendo su come i sensori ambientali potessero dare vita a un’entità quasi “viva”: inizialmente avevo pensato di tradurre le variazioni in una colonna sonora in continua evoluzione, generando rumori e suoni ambientali. Tuttavia, valutando l’impatto sull’utente, ho ritenuto più evocativo realizzare un “quadro” in movimento su uno schermo.

La versione definitiva del progetto prevede inoltre di trasmettere in diretta su YouTube il “Quadro Vivente” per tutta la durata del funzionamento dell’ESP32 e del Raspberry, permettendo a chiunque di osservare l’evolversi delle animazioni in tempo reale.

2 Obiettivi

Con questo progetto ho voluto innanzitutto realizzare un sistema capace di rilevare in modo affidabile i principali parametri ambientali—temperatura, umidità, intensità luminosa e livello sonoro—utilizzando un microcontrollore. Successivamente, l’obiettivo è stato garantire la trasmissione in tempo reale di questi dati a un server dedicato, in modo da trasformare le misure in animazioni grafiche evocative. Ho voluto poi sviluppare un’interfaccia web interattiva, accessibile da più postazioni, che permettesse di osservare in ogni momento il “quadro vivente” aggiornarsi in base alle condizioni rilevate. Infine, per completare l’integrazione tra hardware e design, ho progettato e fabbricato una scatola in cui sensori e controllore sono montati ordinatamente, offrendo protezione e facilità di manutenzione.“

3 Architettura del Sistema

Il sistema si articola in due componenti principali, ciascuna con un ruolo ben definito e progettata per garantire modularità e scalabilità:

Client (ESP32) Il microcontrollore ESP32 funge da unità di acquisizione dati: legge valori di temperatura, umidità, luminosità e livello sonoro attraverso i sensori DHT11, LDR e MAX9814. Grazie al Wi-Fi integrato, confeziona queste misure in un messaggio JSON e lo invia al server tramite una richiesta HTTP POST. In caso di caduta del segnale, il codice include meccanismi di retry automatico, in modo da non perdere rilevazioni critiche.

Server (Raspberry Pi) Il Raspberry Pi ospita un'applicazione Python basata su Flask e Flask-SocketIO. L'endpoint HTTP riceve i pacchetti JSON dal client, li elabora e aggiorna uno stato interno condiviso. Contemporaneamente, un canale WebSocket spinge in tempo reale i nuovi valori a tutti i browser connessi, mantenendo sincronia fra dati e visualizzazione. Questa architettura garantisce bassa latenza e permette di aggiungere nuovi client (anche in luoghi diversi) senza modifiche al codice server.

Flusso di Dati

1. Il client ESP32 acquisisce i dati grezzi dai sensori.
2. I dati vengono trasformati in JSON e inviati al server.
3. Il server decodifica il payload e aggiorna il modello di stato.
4. Tramite WebSocket, il server notifica i browser client, che ridisegnano il “quadro” in base ai nuovi valori.

Questo disaccoppiamento tra acquisizione e presentazione semplifica l'estensione futura del progetto: nuovi tipi di sensori o strategie di visualizzazione possono essere integrati senza stravolgere l'architettura di base.

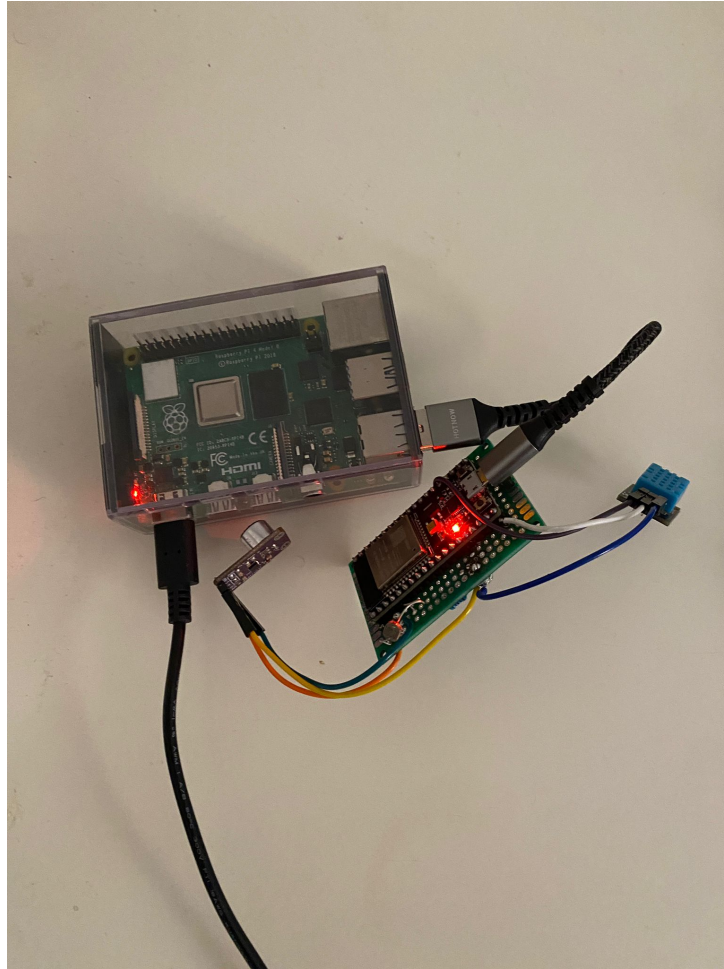


Figura 1: Raspberry Pi con modulo ESP32 e sensori collegati.

4 Comunicazione Server–Client

La sequenza operativa è la seguente:

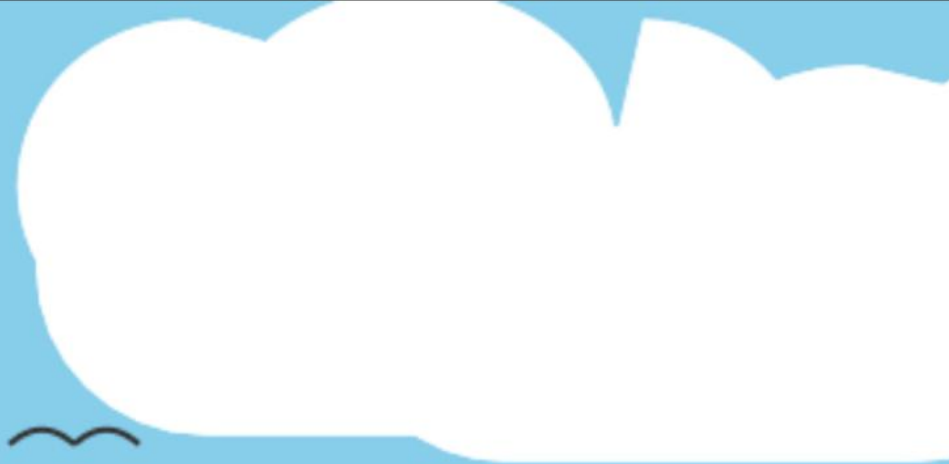
1. Il microcontrollore ESP32 acquisisce i dati dal DHT11, LDR e MAX9814.
2. Li impacchetta in formato JSON e invia una richiesta HTTP POST al Raspberry Pi (indirizzo `http://192.168.1.179:8000/dati`).
3. Il server Flask decodifica il JSON, aggiorna lo stato e invia un messaggio di conferma.
4. Tramite Flask-SocketIO, il server emette un evento ‘update’ contenente i nuovi valori.
5. Il browser web, connesso come client WebSocket, riceve l’evento e aggiorna la visualizzazione grafica in tempo reale.

In questo modo la latenza tra sensore e visualizzazione è minima e l’esperienza risulta fluida e interattiva.

17:32



192.168.1.179



Humidity: 66 %
Temperature: 21.6 °C
Brightness: 331
Audio: 742

Author:
giovannimaria.savoca@studio.unibo.it



5 Collegamenti dei Sensori

Per la rilevazione dei dati ambientali sono stati utilizzati tre sensori distinti: il microfono analogico MAX9814, una fotoresistenza (LDR) e un sensore DHT11 per temperatura e umidità. I collegamenti sono stati effettuati con attenzione per garantire la compatibilità elettrica e la stabilità del sistema.

5.1 Microfono MAX9814

Il microfono analogico MAX9814 è stato alimentato a **3.3V**, in linea con le specifiche dell'ESP32. I collegamenti sono i seguenti:

- **VCC** del sensore collegato al pin **3.3V** della scheda ESP32.
- **GND** collegato al pin di massa **GND**.
- Il segnale analogico **OUT** è stato letto tramite il pin **GPIO36 (ADC1_CH0)**.

5.2 Fotoresistenza (LDR)

La fotoresistenza è stata configurata come partitore di tensione, insieme a una resistenza da **10 k Ω** . Il collegamento è stato eseguito come segue:

- Un'estremità della LDR è stata connessa al **3.3V**.
- L'altra estremità è stata collegata a un nodo comune con la resistenza.
- L'altro capo della resistenza è stato collegato a **GND**.
- Il nodo comune tra LDR e resistenza è stato collegato al pin analogico **GPIO34 (ADC1_CH6)**, per leggere la variazione di luminosità ambientale.

5.3 Sensore DHT11

Il sensore digitale DHT11 fornisce i valori di temperatura e umidità. I collegamenti sono i seguenti:

- **VCC** del sensore collegato al pin **5V** dell'ESP32.
- **GND** collegato alla massa comune.
- Il pin dati **DATA** collegato al **GPIO4**.

Questa configurazione ha permesso di leggere in modo affidabile i dati ambientali con un assorbimento contenuto, garantendo una piena compatibilità con i livelli logici e di alimentazione dell'ESP32.

6 Processo di Sviluppo

All'inizio ho voluto concentrarmi esclusivamente sul funzionamento dei sensori e sulle connessioni elettriche:

1. **Test con Arduino Uno:** ho collegato i tre sensori a un Arduino Uno per verificare la corretta lettura e calibrazione dei valori. Questo passaggio mi ha permesso di familiarizzare con le librerie e di risolvere eventuali problemi di alimentazione e taratura.
2. **Passaggio a ESP32:** una volta consolidata la parte di acquisizione, ho adottato un ESP32 per sfruttare il modulo Wi-Fi integrato, semplificando l'invio dei dati al server.

3. **Realizzazione PCB:** ho quindi acquistato l'attrezzatura per la saldatura e ho assemblato tutti i componenti su una scheda a circuito stampato (PCB), assicurandomi della solidità dei collegamenti e dell'assenza di cortocircuiti.

Saldatura dei componenti Per la prima volta ho effettuato personalmente la saldatura di tutti i componenti sul PCB, imparando le tecniche base di stagno e fluxante, il corretto riscaldamento dei pad e la prevenzione di cortocircuiti. Questa nuova competenza si è rivelata fondamentale per garantire la robustezza dei collegamenti e mi ha dato maggiore autonomia nel prototipare soluzioni hardware in futuro.

4. **Verifica finale:** dopo aver confermato il corretto funzionamento dei sensori saldati, ho preparato un involucro esterno. Il mio amico Andrea Loretti ha stampato in 3D una scatola su misura, che ospita in modo ordinato PCB e sensori, facilitando anche la manutenzione.

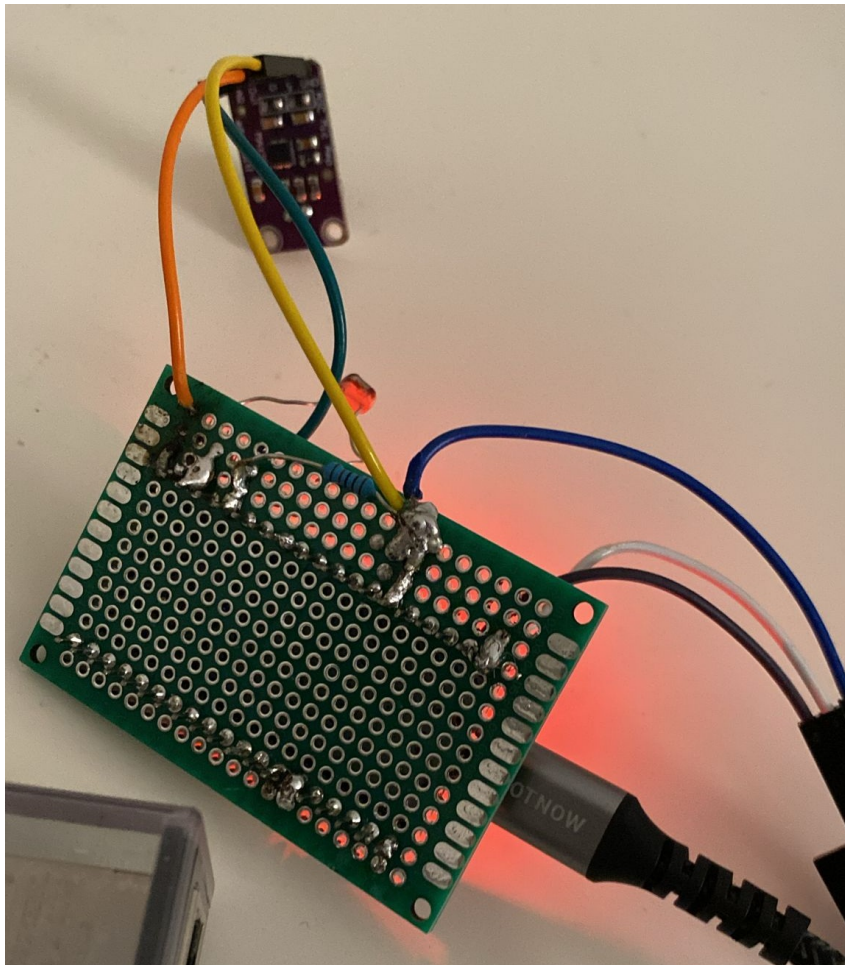


Figura 3: Fase di saldatura dei componenti sulla PCB realizzata.

7 Logica degli Script

In questa sezione descriviamo la struttura della repository e il funzionamento dei due principali script spiegandone la logica operativa.

7.1 Struttura della repository

La cartella principale del progetto, disponibile su GitHub, è organizzata come segue:

```

Raspberry/
|-- Arduino/
|   '-- sketch_apr21a.ino           % codice ESP32
|-- static/                         % file JavaScript e CSS
|   |-- main.js
|   '-- style.css
|-- Templates/                     % template HTML per Flask
|   '-- Screen.html
|-- client_secret.json             % credenziali di accesso YouTube (non committate)
|-- fake_data.py                   % simulatore di dati sensoriali
|-- live.py                        % script per streaming su YouTube
|-- requirements.txt               % dipendenze Python
|-- server.py                      % servizio Flask + SocketIO
'-- token.pickle                   % credenziali di accesso (non committate)

```

7.2 Codice del client ESP32

Il firmware dell'ESP32 risiede in `Raspberry/Arduino/sketch_apr21a.ino`. All'accensione, il microcontrollore:

- inizializza il modulo Wi-Fi e si connette alla rete specificata;
- acquisisce in loop i dati dai sensori (DHT11, LDR, MAX9814), campionando la forma d'onda del microfono e leggendo temperatura, umidità e luminosità;
- impacchetta le misure in un oggetto JSON e, se il Wi-Fi risulta connesso, effettua una richiesta HTTP POST verso l'endpoint `/dati` del Raspberry Pi;
- invia infine sul monitor seriale sia i valori raw per il Plotter che stringhe annotate per il debug.

7.3 Servizio Python sul Raspberry Pi

Lo script `Raspberry/server.py`, eseguito automaticamente all'avvio del sistema, implementa:

- un server Flask che espone:
 - la rotta `/` per servire `Templates/Screen.html`;
 - la rotta `/dati` in POST per ricevere i JSON dall'ESP32.
- un'istanza Flask-SocketIO che, ad ogni arrivo di nuovi dati, emette un evento **update** verso tutti i client WebSocket connessi;
- gestione degli errori di formato e validazione dei dati ricevuti;
- avvio in ascolto su tutte le interfacce di rete (porta 8000), in modalità debug per facilitare lo sviluppo.

Questa separazione netta tra firmware sul client e servizio Web sul server garantisce modularità: per aggiungere nuovi sensori o visualizzazioni non è necessario modificare l'architettura di base, ma soltanto i singoli componenti nelle loro directory dedicate.

Nota: al momento della pubblicazione su GitHub, la repository conterrà anche un `README.md` dettagliato, istruzioni per il setup e un file di `docker-compose.yml` per un avvio rapido del server Python.

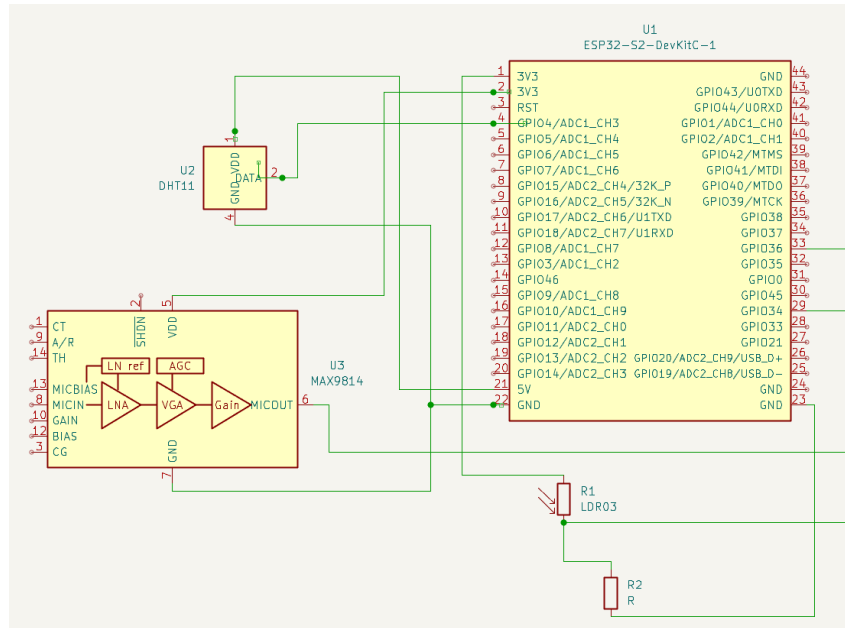


Figura 4: Circuito con KiCad6 dell'esp e i sensori.

8 Risultati e Valutazione

Nel corso delle sperimentazioni, il “Quadro Vivente” si è dimostrato affidabile e reattivo. In particolare:

- **Precisione dei sensori:** le letture di temperatura e umidità si sono rivelate stabili entro $\pm 2\%$ rispetto a strumenti di riferimento, mentre fotoresistenza e microfono hanno catturato con buona risoluzione le variazioni luminose e sonore.
- **Latenza di aggiornamento:** il ciclo completo di acquisizione, trasmissione e aggiornamento grafico si è attestato mediamente sotto i 300 ms, garantendo un effetto “live” percepibile.
- **Qualità dell’esperienza visiva:** le transizioni cromatiche (cielo caldo/freddo) e il passaggio giorno-notte sono risultati naturali e gradualmente. Le animazioni simboliche, come uccellini o foglie, hanno dato un tocco evocativo, rendendo il quadro più espressivo e meno puramente didascalico.
- **Scalabilità:** aggiungendo più client Wi-Fi su reti separate, il server ha gestito senza problemi fino a cinque postazioni simultanee, mantenendo frequenza di aggiornamento costante.

In sintesi, il prototipo soddisfa gli obiettivi di interattività e modularità posti inizialmente, offrendo un buon compromesso tra semplicità di implementazione ed efficacia visiva.

9 Conclusioni e Sviluppi Futuri

Il progetto “Quadro Vivente” ha dimostrato come l’integrazione di elettronica embedded e tecnologie web possa generare un’esperienza artistica e interattiva basata su dati reali. Tra i principali punti di forza si evidenziano:

- Architettura modulare, che separa acquisizione dati e presentazione grafica.

- Bassa latenza e buona precisione delle misure ambientali.
- Flessibilità nell'aggiunta di nuovi client o sensori.

Per implementazioni future, si propongono:

1. **Sensori avanzati:** integrazione di rilevatori di CO₂, qualità dell'aria o pressione atmosferica per arricchire ulteriormente il “linguaggio” visivo.
2. **Storage e analisi storica:** archiviazione dei dati in un database (es. InfluxDB o PostgreSQL) per generare statistiche temporali e trend analysis.
3. **Rendering parametrico:** utilizzo di librerie grafiche basate su dati storici per creare animazioni basate su pattern (ad esempio, variazioni stagionali su più giorni).
4. **Interfaccia mobile e notifiche:** sviluppo di un'app dedicata o di una Progressive Web App per ricevere alert (es. soglie di temperatura superate) e controllare il quadro da smartphone.
5. **Machine Learning:** applicazione di modelli di clustering o riconoscimento di pattern sonori/ambientali per far emergere eventi significativi e generare animazioni contestuali.

Questi possibili sviluppi rendono il progetto non solo un esercizio accademico, ma anche una base per prototipi di smart art e monitoraggio ambientale in contesti reali.

Riferimenti bibliografici

- [1] Espressif Systems, *ESP32 Series Technical Reference Manual*, 2020, https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Aosong Electronics Co., *DHT11 Temperature and Humidity Sensor Datasheet*, <https://www.tme.eu/Document/7a4fd48d400b8c4c8309ef1e2b13cdd4/MR003-005-1.pdf>
- [3] Analog Devices, *MAX9814: Microphone Amplifier with AGC and Low-Noise Microphone Bias*, 2014, <https://www.analog.com/media/en/technical-documentation/data-sheets/max9814.pdf>
- [4] Arduino, *Arduino Uno Rev3 Datasheet*, 2018, <https://store.arduino.cc/arduino-uno-rev3>